the displayed data changes. Very broadly speaking, there are a few ways to handle this change. Clicking on a channel could trigger an entire page refresh, making a normal HTTP request to the server and redrawing the entire page. This would usually cause the page to flicker, leading to a poor user experience, although in theory, when using default Rails, the Turbo Drive part of the Turbo library prevents flickering. This is the solution with the least JavaScript.

Clicking on a channel could trigger sending a request to the server, where the server handles the rendering logic and returns the HTML of the part of the page that changes. The client side integrates that HTML into the page. There is a little bit of UI cleanup, such as changing the active status of channels in the sidebar. This would either be done in the client-side code, or, alternately, the server could return a JavaScript script that both updated the HTML and did the UI cleanup or return multiple chunks of HTML that manage the cleanup. This is the JavaScript interaction pattern that many early Rails applications used and which you see a new version of in the Hey[25] email app and other apps that use Turbo Frames and Turbo Streams.

Clicking on a channel could also trigger sending a request to the server that returns data in JSON format. The JavaScript code is responsible for using that data to update, which different frameworks handle in different ways. The data update would trigger changes to the DOM, and the specific updated DOM elements would be redrawn. This is the interaction pattern that React and most JavaScript applications use these days.

When we switch to the profile page, we have the same options, plus we now have one more kind of state to deal with—the name of the entire page that we are looking at. In an older Rails app, switching to a new profile page would be a request to a different URL, and the routing table in the Rails app would handle the decision of what view templates to render and send back to the browser.

In an SPA, the routing happens on the client, and the client's routing table intercepts the click, determines what components are displayed, and what data needs to be retrieved from the server to render that data. This routing is slightly more complicated because it splits one step (call the server) into two (navigate